

Algorithmen und Datenstrukturen



Ziele der heutigen Lerneinheit

Sie verstehen...

- warum B-Bäume für externe Speicher entwickelt wurden und welche Probleme sie lösen
- wie die Ordnung eines B-Baums die Knotenstruktur bestimmt
- wie die Balancierung des Baums beim Einfügen (Splitten) und Löschen (Verschmelzen/Verschieben) aufrechterhalten wird

Sie können...

- die Suche in einem B-Baum Schritt für Schritt nachvollziehen und die Effizienz in O-Notation angeben
- einen Schlüssel korrekt in einen B-Baum einfügen, inklusive dem Aufteilen voller Knoten
- einen Schlüssel aus einem B-Baum löschen und dabei Unterlauf-Situationen durch Verschieben oder Verschmelzen von Knoten behandeln

B-Bäume

B-Bäume

Speicherstruktur für externe Speicher

- Die bisher betrachteten Bäume gehen davon aus, dass die gesamte Datenmenge in den Hauptspeicher passt
- Zwar schafft eine virtuelle Speicherverwaltung die Illusion eines nahezu unbegrenzten Hauptspeichers, allerdings auf Kosten von häufigem Ein- und Auslagern der referenzierten Speicherseiten
- Der Zugriff auf externe Speicher ist um Größenordnungen langsamer als der Zugriff auf den Hauptspeicher. Daher wird i.d.R. immer eine Seite geladen bzw. gespeichert
- Ziel muss es daher bei großen Datenmengen – neben der Effizienz der Algorithmen – mit wenigen Zugriffen auf externe Speicher auszukommen

B-Bäume

Speicherstruktur für externe Speicher

- Bei einem B-Baum entspricht ein Knoten einer „Seite“ auf dem externen Speicher
- Daher sind Knoten eines B-Baums größer und enthalten mehr als einen Schlüsselwert
- Die Ordnung eines B-Baums legt die Größe der Knoten fest. Ein B-Baum der Ordnung m
 - besitzt Knoten mit **maximal** $2m - 1$ Schlüsselwerten
 - besitzt (mit Ausnahme der Wurzel) Knoten mit **mindestens** $m - 1$ Schlüsselwerten
- Dennoch ist ein B-Baum ein Suchbaum, in dem durch Schlüsselvergleiche ein Abstieg im Baum gesteuert wird
- Entwickelt wurde der B-Baum von R. Bayer und E. McCreight
- **B** steht für balanciert, breit, buschig, Bayer, nicht für binär

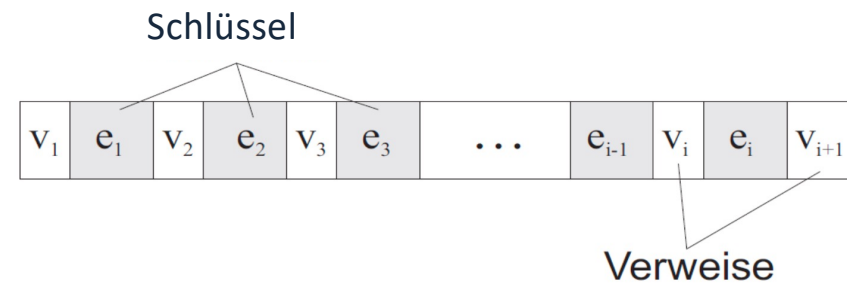
B-Bäume

Aufbau eines Knotens

Die Schlüsselwerte e innerhalb eines Knotens sind sortiert.

Zwischen den i Schlüsselwerten gibt es Verweise auf $i+1$ Unterbäume.

Alle Schlüssel im linken Unterbaum zu einem Schlüssel e_j sind kleiner und alle Schlüssel im rechten Unterbaum zu einem Schlüssel e_j sind größer als e_j .



Die Datenstruktur für einen Knoten:

- n : Aktuelle Anzahl von Schlüsseln im Knoten
- $keys$: Array/Vektor mit den Schlüsseln des Knoten
- $refs$: Array/Vektor mit den Verweisen des Knoten
- $isLeaf$: zeigt an, ob es ein Blattknoten ist

B-Bäume

Aufbau des Baums

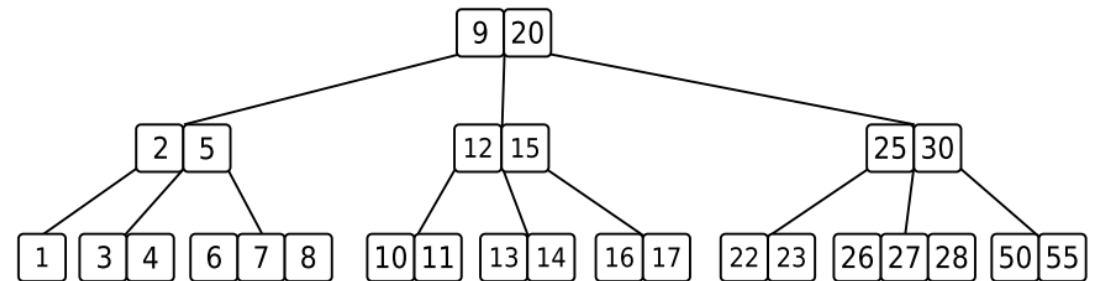
Alle Blattknoten des B-Baums befinden sich in gleicher Tiefe

Die Tiefe ist gleich der Höhe h des Baums

Bei einem B-Baum der Ordnung m gilt

$$h \leq \log_m \frac{n+1}{2}$$

Beispiel für einen Baum der Ordnung 2:



B-Bäume

Anlegen eines leeren Knotens

Beim Anlegen eines leeren Knotens werden die Attribute gemäß des gezeigten Pseudocodes initialisiert.

MAKE-NODE(m)

```
1:  $n \leftarrow 0$   
2:  $keys \leftarrow \mathbf{new\ Array}[2m - 1]$   $\triangleright [0 \dots 2m - 2]$   
3:  $refs \leftarrow \mathbf{new\ Array}[2m]$   $\triangleright [0 \dots 2m - 1]$   
4:  $isLeaf \leftarrow \mathbf{true}$ 
```

B-Bäume

Suche in B-Bäumen

Das Suchen in B-Bäumen ist eine Kombination von

- Verfolgen von Verweisen (wie beim binären Suchbaum)
- Suche in einem sortierten Array

BTREE-SEARCH(*start*, *key*)

```
1:  $i \leftarrow 0$ 
2: while  $i < start.n$  and  $key > start.keys[i]$  do
3:    $i \leftarrow i + 1$ 
4: end while
5: if  $i < start.n$  and  $key = start.keys[i]$  then
6:   return start
7: end if
8: if start.isLeaf then
9:   return NIL
10: end if
11: return BTREE-SEARCH(start.refs[i], key)
```

B-Bäume

Suche in B-Bäumen

Effizienzbetrachtung

Die Anzahl der Zugriffe auf das externe Medium ist gleich der Anzahl der besuchten Knoten:

$$O(\log_m n)$$

Verwendet man eine einfache lineare Suche in den Knoten, so ergibt sich ein Gesamtaufwand von

$$O(\log_m n \cdot m)$$

BTREE-SEARCH(*start*, *key*)

```
1:  $i \leftarrow 0$ 
2: while  $i < start.n$  and  $key > start.keys[i]$  do
3:    $i \leftarrow i + 1$ 
4: end while
5: if  $i < start.n$  and  $key = start.keys[i]$  then
6:   return start
7: end if
8: if start.isLeaf then
9:   return NIL
10: end if
11: return BTREE-SEARCH(start.refs[i], key)
```

B-Bäume

Aufspalten eines B-Baum-Knotens

Beim Einfügen eines neuen Schlüssels in einen B-Baum kann es zu einem Überlauf in einem Knoten kommen, wenn dort bereits $2m-1$ Schlüssel gespeichert sind.

In diesem Fall wird der Knoten aufgespaltet; die Tiefe des Baums ändert sich dadurch nicht.

Im Elternknoten wird ein zusätzlicher Schlüssel eingefügt, damit ein weiterer Verweis auf den neuen Knoten möglich ist.

B-Bäume

Aufspalten eines B-Baum-Knotens

Ausgehend von einem *parent*-Knoten spaltet der nebenstehende Algorithmus den *i*. Kindknoten in einem B-Baum der Ordnung *m* auf.

Voraussetzungen:

- Der Kindknoten besitzt vorher mindestens $2 \cdot (m - 1) + 1 = 2m - 1$ Schlüssel (d.h. er ist voll), damit in den Splittknoten die Mindestanzahl erreicht wird und ein weiterer Schlüssel in den Elternknoten wandern kann
- Der Elternknoten ist noch nicht so voll, dass kein weiterer Schlüssel aufgenommen werden kann

BTREE-SPLIT-CHILD(*parent*, *i*, *m*)

```

1: child  $\leftarrow$  parent.refs[i]
2: h  $\leftarrow$  MAKE-NODE(m)
3: h.isLeaf  $\leftarrow$  child.isLeaf
4: h.n  $\leftarrow$  m - 1
5: for j  $\leftarrow$  0 to m - 2 do
6:   h.keys[j]  $\leftarrow$  child.keys[j + m]
7: end for
8: if not h.isLeaf then
9:   for j  $\leftarrow$  0 to m - 1 do
10:    h.refs[j]  $\leftarrow$  child.refs[j + m]
11:   end for
12:   for j  $\leftarrow$  m to child.n do
13:    child.refs[j]  $\leftarrow$  NIL
14:   end for
15: end if
16: child.n  $\leftarrow$  m - 1
17: for j  $\leftarrow$  parent.n downto i + 1 do
18:   parent.refs[j + 1]  $\leftarrow$  parent.refs[j]
19:   parent.keys[j]  $\leftarrow$  parent.keys[j - 1]
20: end for
21: parent.refs[i + 1]  $\leftarrow$  h
22: parent.keys[i]  $\leftarrow$  child.keys[m - 1]
23: parent.n  $\leftarrow$  parent.n + 1

```

B-Bäume

Aufspalten eines B-Baum-Knotens

Effizienzbetrachtung

Es wird nur eine konstante Anzahl von Knoten bearbeitet ($parent, child, h$), daher ist der Aufwand für den Zugriff auf externen Speicher

$$O(1)$$

Alle Schleifen durch m begrenzt, somit insgesamt

$$O(m)$$

BTREE-SPLIT-CHILD($parent, i, m$)

```

1:  $child \leftarrow parent.refs[i]$ 
2:  $h \leftarrow \text{MAKE-NODE}(m)$ 
3:  $h.isLeaf \leftarrow child.isLeaf$ 
4:  $h.n \leftarrow m - 1$ 
5: for  $j \leftarrow 0$  to  $m - 2$  do
6:    $h.keys[j] \leftarrow child.keys[j + m]$ 
7: end for
8: if not  $h.isLeaf$  then
9:   for  $j \leftarrow 0$  to  $m - 1$  do
10:     $h.refs[j] \leftarrow child.refs[j + m]$ 
11:   end for
12:   for  $j \leftarrow m$  to  $child.n$  do
13:     $child.refs[j] \leftarrow \text{NIL}$ 
14:   end for
15: end if
16:  $child.n \leftarrow m - 1$ 
17: for  $j \leftarrow parent.n$  downto  $i + 1$  do
18:    $parent.refs[j + 1] \leftarrow parent.refs[j]$ 
19:    $parent.keys[j] \leftarrow parent.keys[j - 1]$ 
20: end for
21:  $parent.refs[i + 1] \leftarrow h$ 
22:  $parent.keys[i] \leftarrow child.keys[m - 1]$ 
23:  $parent.n \leftarrow parent.n + 1$ 

```

B-Bäume

Einfügen in einen B-Baum

Der neue Schlüssel wird grundsätzlich in ein Blatt eingetragen.

Beim rekursiven Abstieg zum betroffenen Blatt im Baum wird geprüft, ob in einen vollen Knoten abgestiegen werden soll. Falls ja, wird dieser gespalten.

Von oben kommend wird damit sichergestellt, dass

- der von der Spaltung nach oben durchgereichte Schlüssel nicht auf einen vollen Knoten trifft
- das Blatt noch einen Schlüssel aufnehmen kann

B-Bäume

Einfügen in einen B-Baum

Zunächst muss der Sonderfall betrachtet werden, dass die Wurzel bereits voll ist.

Es erfolgt dann ein Splitt und der Baum wächst nach oben.

Somit bleibt er stets balanciert.

BTREE-INSERT(T, key)

```
1:  $r \leftarrow T.root$ 
2: if  $r.n = 2m - 1$  then
3:    $h \leftarrow \text{MAKE-NODE}(T.m)$ 
4:    $T.root \leftarrow h$ 
5:    $h.isLeaf \leftarrow \text{false}$ 
6:    $h.refs[0] \leftarrow r$ 
7:    $h.n \leftarrow 0$ 
8:   BTREE-SPLIT-CHILD( $h, 0, T.m$ )
9:   BTREE-INSERT-IN-NODE( $h, key, T.m$ )
10: else
11:   BTREE-INSERT-IN-NODE( $r, key, T.m$ )
12: end if
```

B-Bäume

Einfügen in einen B-Baum

Effizienzbetrachtung

Anzahl der Zugriffe auf das externe Medium:

$$O(\log_m n)$$

In jedem Knoten max. m Vergleiche, somit insgesamt:

$$O(\log_m n \cdot m)$$

BTREE-INSERT-IN-NODE(*start*, *key*, *m*)

```

1: i ← start.n
2: if start.isLeaf then
3:   while i ≥ 1 and key < start.keys[i − 1] do
4:     start.keys[i] ← start.keys[i − 1]
5:     i ← i − 1
6:   end while
7:   start.keys[i] ← key
8:   start.n ← start.n + 1
9: else
10:  j ← 0
11:  while j < start.n and key > start.keys[j] do
12:    j ← j + 1
13:  end while
14:  if start.refs[j].n = 2m − 1 then
15:    BTREE-SPLIT-CHILD(start, j, m)
16:    if key > start.keys[j] then
17:      j ← j + 1
18:    end if
19:  end if
20:  BTREE-INSERT-IN-NODE(start.refs[j], key, m)
21: end if

```

B-Bäume

Löschen aus einem B-Baum

Der zu löschende Schlüssel muss zunächst gefunden werden; er kann sich in einem inneren Knoten oder einem Blatt befinden.

Mit dem Entfernen des Schlüssels aus einem inneren Knoten verschwindet auch eine Verzeigerung zu einem möglichen Unterbaum.

Beim Entfernen des Schlüssels kann ein Unterlauf auftreten, wenn die Anzahl der Schlüssel im Knoten den Wert $m-1$ unterschreitet.

B-Bäume

Löschen aus einem B-Baum

Lösungsansatz

Analog zum Einfügen, wo beim rekursiven Abstieg sichergestellt wurde, dass die besuchten Knoten noch Schlüssel aufnehmen können (und ggf. gesplittet wurde)

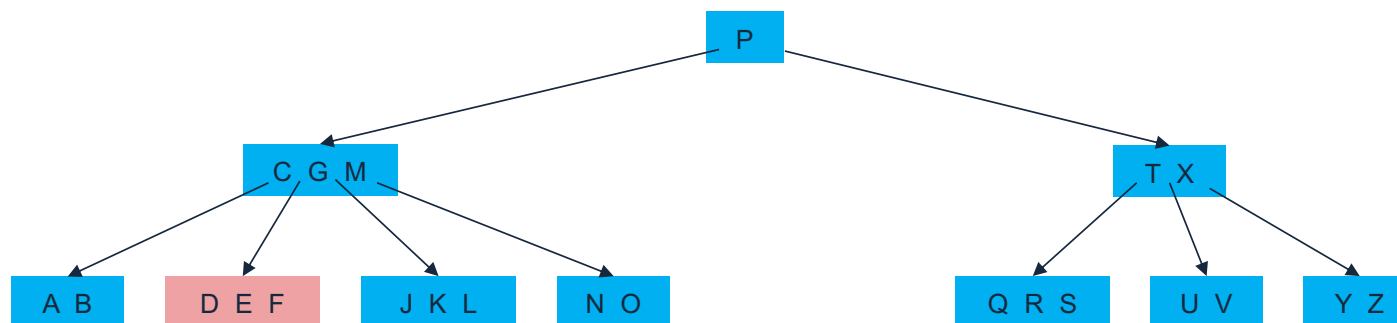
Jetzt wird beim rekursiven Abstieg sichergestellt, dass die besuchten Knoten noch Schlüssel abgeben können. Falls das nicht möglich ist, wird

- vom linken oder rechten Geschwisterknoten ein Schlüssel verschoben, sofern einer der Geschwister Schlüssel abgeben kann
- der Knoten mit einem Geschwister verschmolzen

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (F) aus einem Blatt

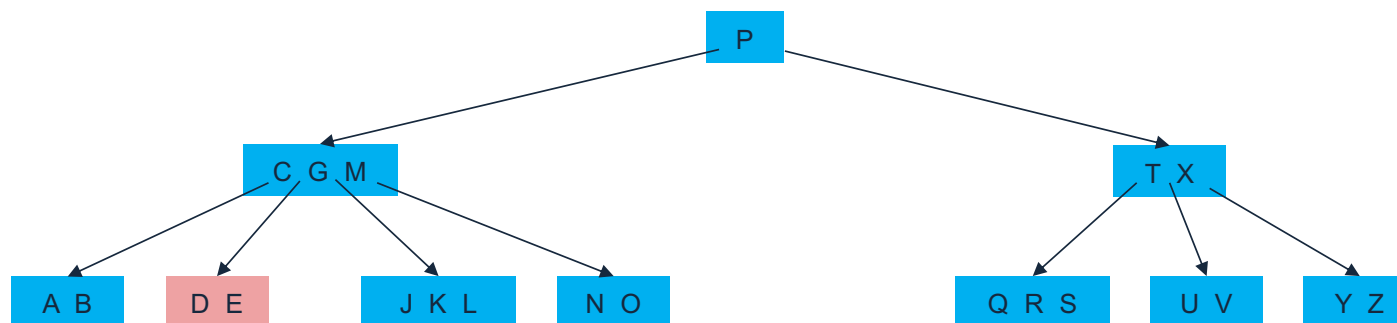


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m - 1 = 2$)
- Keine Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (F) aus einem Blatt

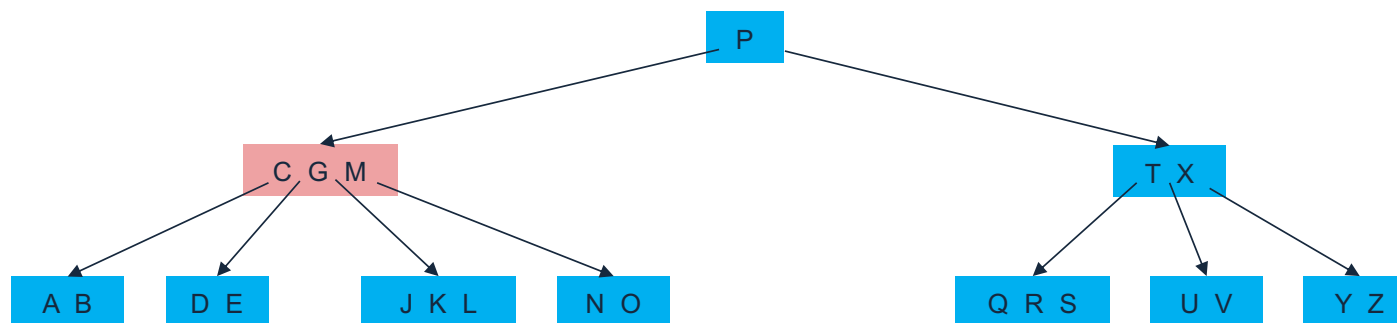


- Der Schlüssel kann einfach entfernt werden

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (M) aus einem inneren Knoten

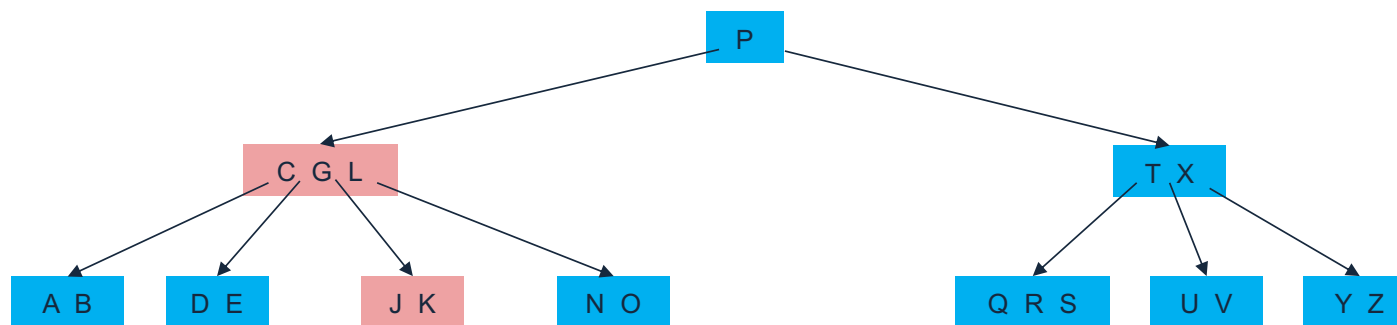


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m - 1 = 2$)
- Es gibt Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (M) aus einem inneren Knoten

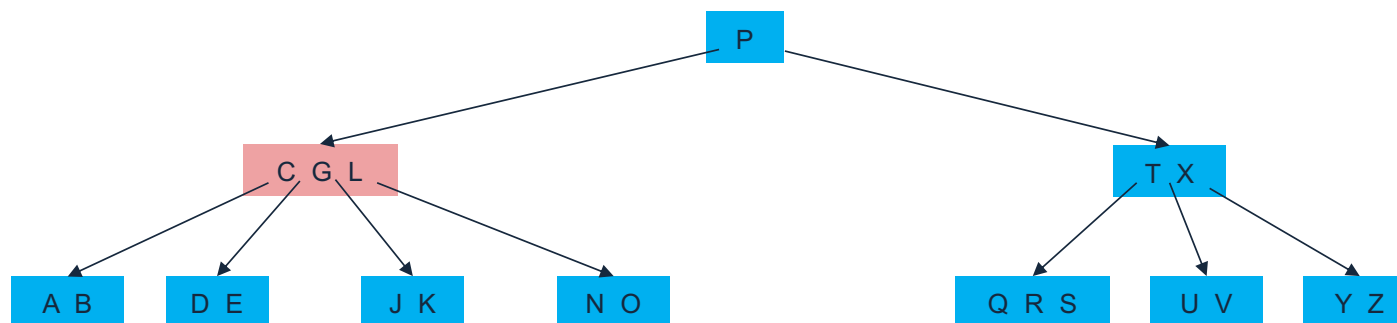


- Der Vorgänger von M rückt nach oben und nimmt den Platz ein
- Der Vorgänger L hat keine Unterbäume und kann verschoben werden

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (G) aus einem inneren Knoten

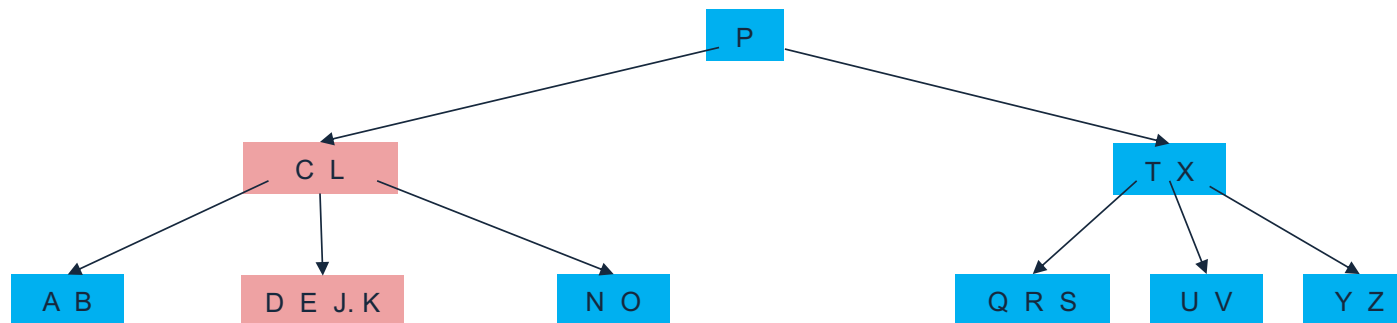


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m-1=2$)
- Es gibt Unterbäume
- Vorgänger kann nicht aufsteigen, weil Mindestgröße unterschritten würde

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (G) aus einem inneren Knoten

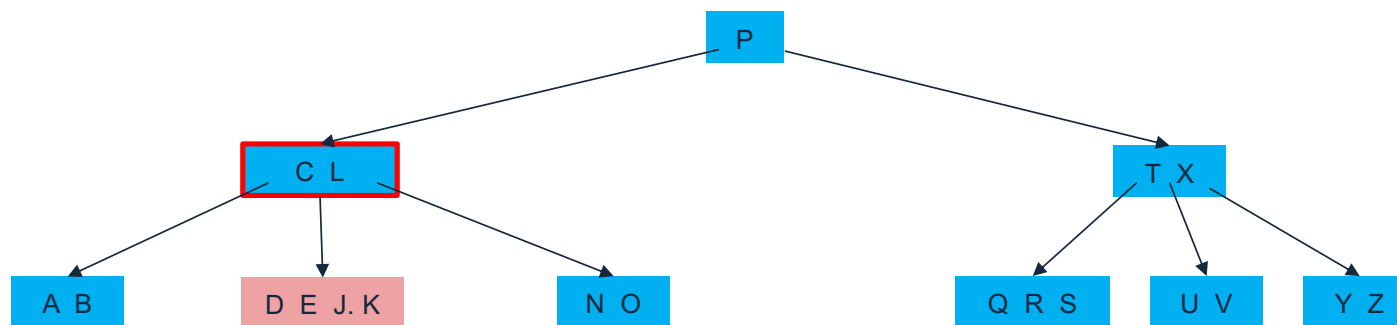


– Verschmelzen der beiden Nachfolger

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (D) aus einem Blatt

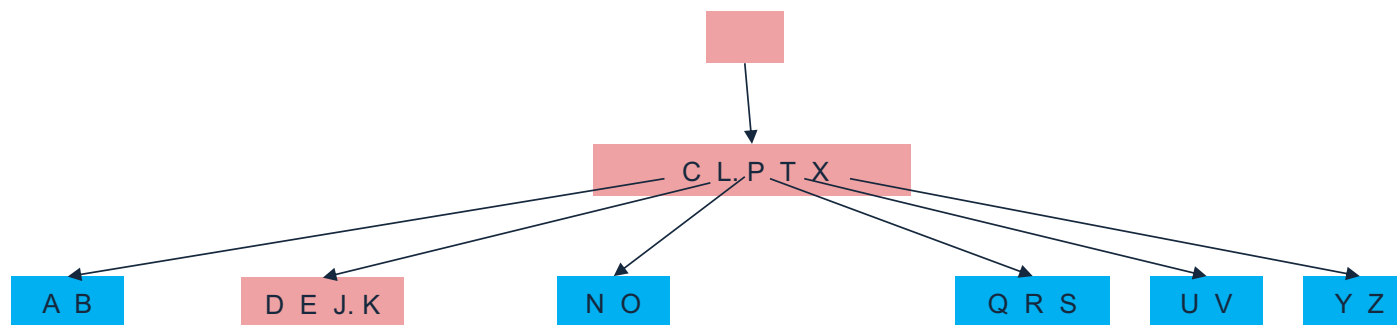


- Der Knoten „C L“ kann keinen Schlüssel mehr abgeben
- Kein Unterschreiten der minimalen Knotengröße ($m-1=2$)
- Es gibt keine Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (D) aus einem Blatt

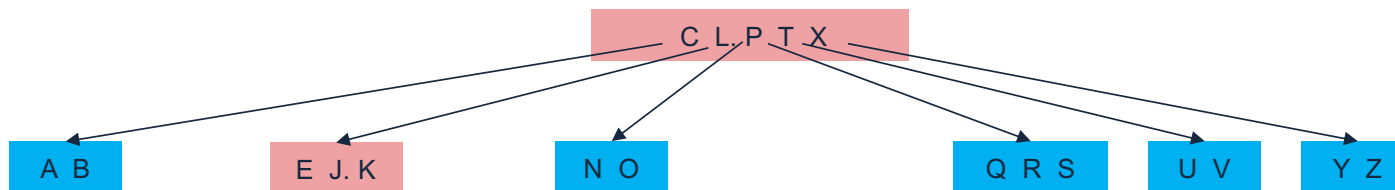


- Verschmelzen von „C L“ mit dem Geschwister „T X“ unter Einbeziehung des ordnenden Schlüssels aus dem Elternknoten

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (D) aus einem Blatt

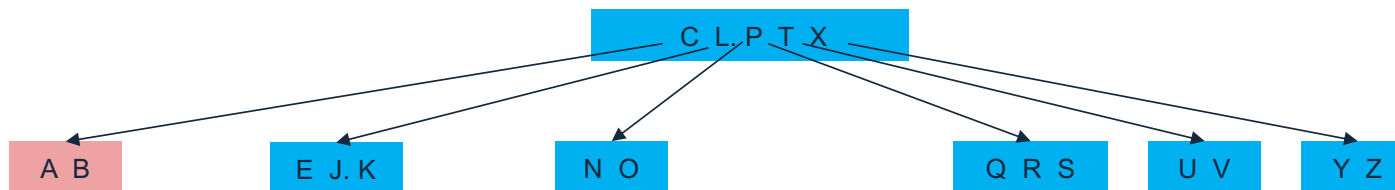


- Danach einfaches Löschen eines Schlüssels aus dem Blatt
- Außerdem kann die leere Wurzel entfernt werden; die Höhe des Baums reduziert sich

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (B) aus einem Blatt

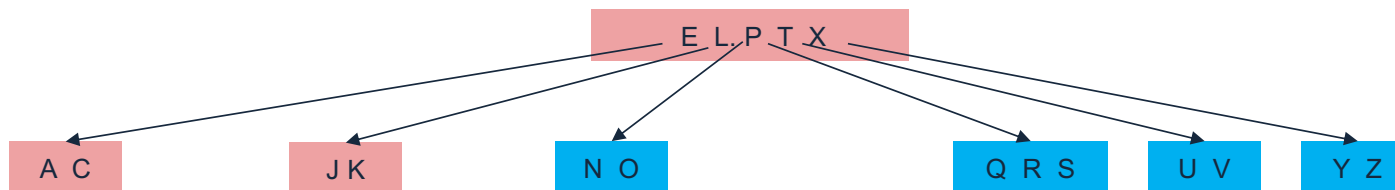


- Alle Knoten auf dem Weg können noch Schlüssel abgeben
- Minimalen Knotengröße ($m-1=2$) wird unterschritten
- Keine Unterbäume

B-Bäume

Beispiel: Löschen aus einem B-Baum der Ordnung 3

Entfernen eines Schlüssels (B) aus einem Blatt



- Verschieben eines Knotens aus dem Geschwisterknoten über den Elternknoten

B-Bäume

Relevanz von B-Bäumen

B-Bäume werden in vielen Dateisystemen und Datenbanken eingesetzt, wie z .B.:

- Windows: NTFS
- Mac: HFS, HFS+, APFS
- Linux: ReiserFS, XFS, Ext3FS, JFS
- Datenbanken: Oracle, DB2, MS SQL, INGRES, ...

Vielen Dank für die Aufmerksamkeit!